1. Беру пайплайн из предыдущего задания, но теперь докер файл создаю не на лету в пайплайне, а в папке проекта. Для этого скачиваю git репозиторий с Github

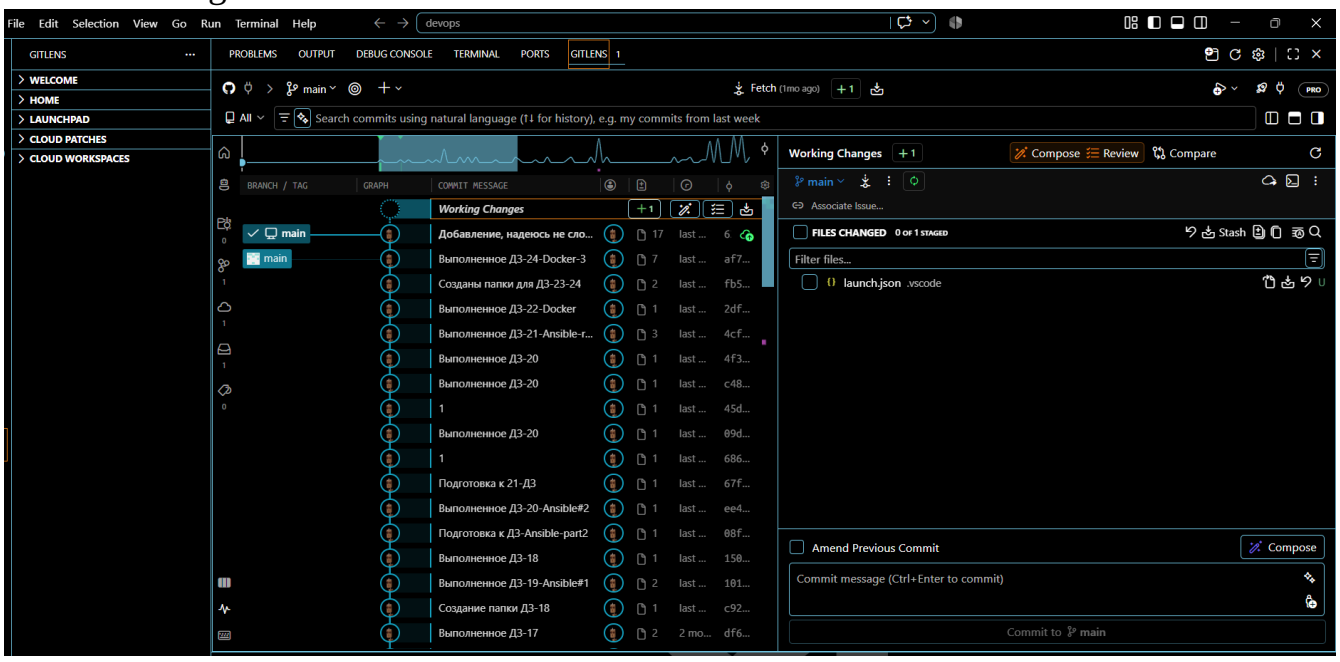
```
anastasia@seigneur-puppy:~$ git clone https://github.com/SeigneurPuppy1/jenkins.git
Cloning into 'jenkins'...
remote: Enumerating objects: 39, done.
remote: Counting objects: 100% (39/39), done.
remote: Compressing objects: 100% (19/19), done.
remote: Total 39 (delta 6), reused 34 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (39/39), 6.25 KiB | 6.25 MiB/s, done.
Resolving deltas: 100% (6/6), done.
```

```
anastasia@seigneur-puppy:~/jenkins$ ll
total 20
drwxrwxr-x  4 anastasia anastasia 4096 Aug  4 16:01 ./
drwxr-x--- 10 anastasia anastasia 4096 Aug  4 16:01 ../
drwxrwxr-x  8 anastasia anastasia 4096 Aug  4 16:01 .git/
-rw-rw-r--  1 anastasia anastasia    8 Aug  4 16:01 .gitignore
-rw-rw-r--  1 anastasia anastasia    0 Aug  4 16:01 README.md
drwxrwxr-x  4 anastasia anastasia 4096 Aug  4 16:01 simple-java-maven-app/
```

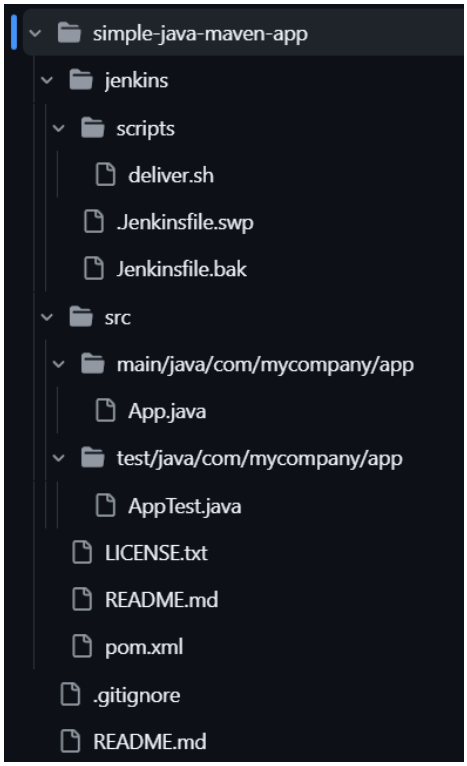
2. И в папке simple-java-maven-app создаю Dockerfile

```
anastasia@seigneur-puppy:~/jenkins/simple-java-maven-app$ cat Dockerfile
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

3. VScode у меня был установлен и до этого, как и поставлен Gitlens, например, также был и git



4. Репозиторий github с моим проектом и кодом тоже существует. Вот как выглядит структура репозитория:

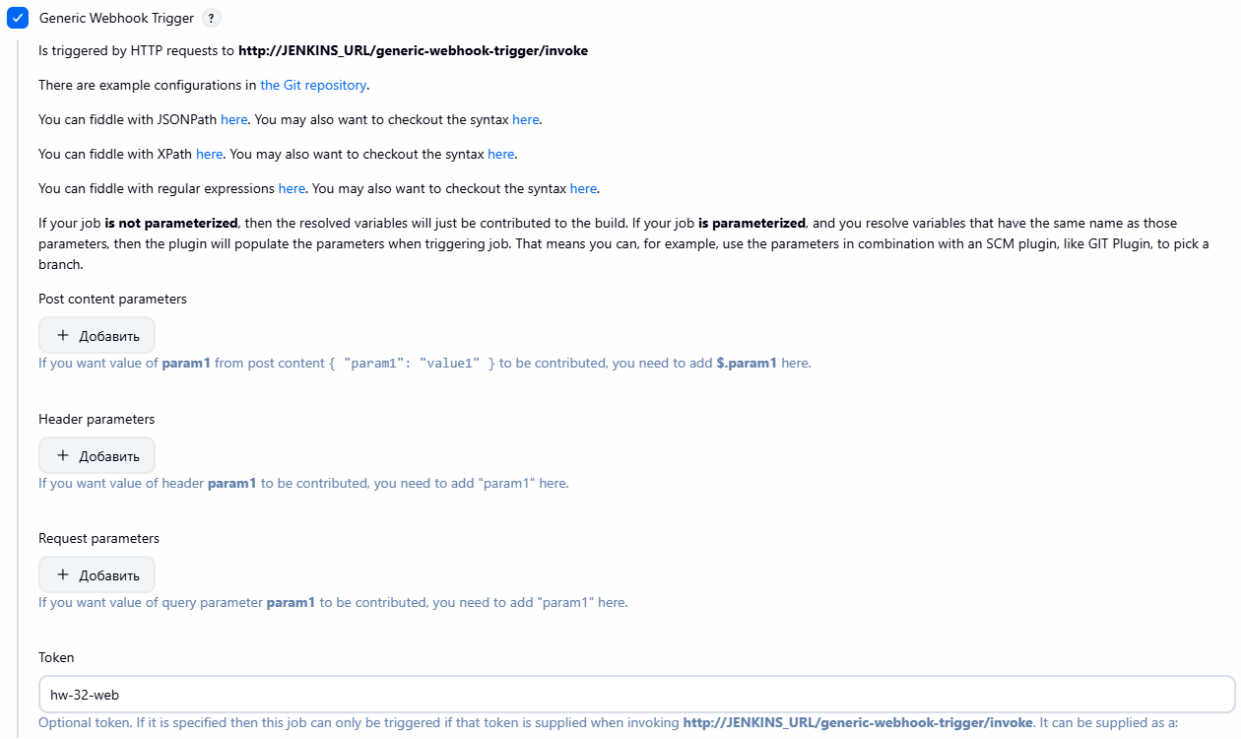


5. Произведена установка расширения для Java, также для проверки синтаксиса пайплайна

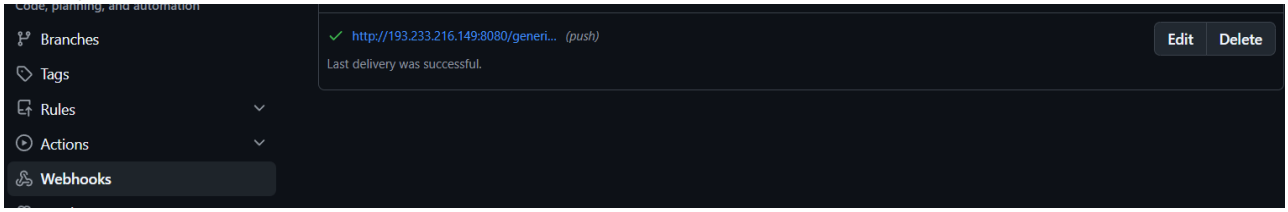




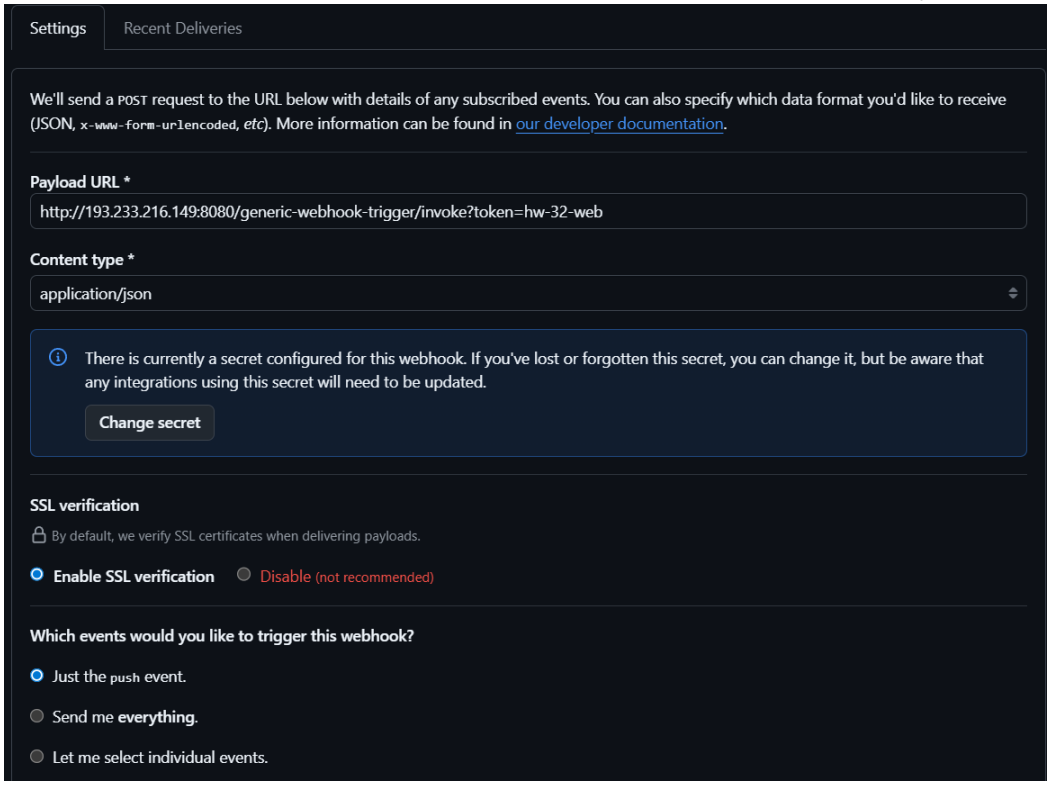
6. Настроила webhook. В самом дженкинсе выбрала пункт добавления веб хука



7. В самом гитхабе перехожу в настройки репозитория, выбираю пункт добавления вебхука



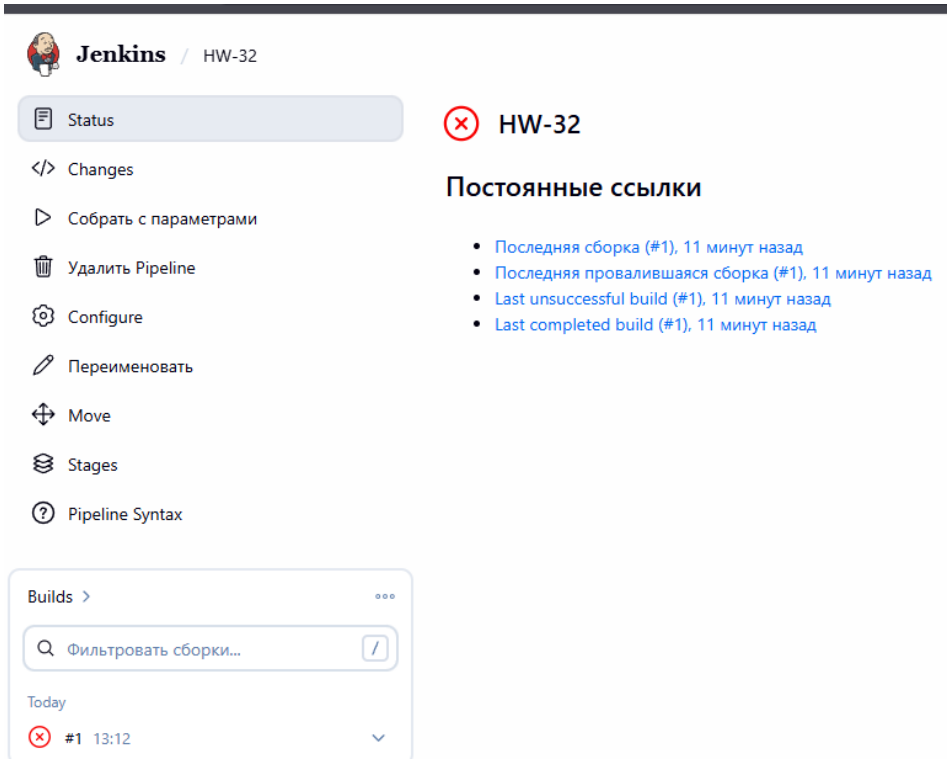
8. Заполняю вот так, токен придумываю сама (hw-32-web), его потом добавляю, как в пункте 6. Настроенный webhook будет работать после push. То есть как только будут запушены изменения в репозиторий проекта github будет оповещать дженкинс об изменениях и автоматически запускать джобу



9. Вот я запушила изменения в репозитории, добавила Dockerfile:

```
anastasia@seigneur-puppy:~/jenkins$ git push git@github.com:SeigneurPuppy1/jenkins.git
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 445 bytes | 445.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:SeigneurPuppy1/jenkins.git
   6d421e3..aadfe11  jenkins -> jenkins
```

10. И вот он автоматически запускает джобу. Пока отработала с ошибкой, так как еще недоделана



11. Вот готовый пайплайн, изменено было только создания докер образа, теперь он не в пайплайне, а в репозитории проекта:

```
pipeline {
    agent any
    parameters {
        string(name: 'server', defaultValue: '62.60.246.108', description: 'IP сервера для запуска приложения')
        string(name: 'APP_NAME', defaultValue: 'maven-app', description: 'Название приложения')
        string(name: 'APP_VER', defaultValue: 'v01', description: 'Версия приложения')
        string(name: 'DOCKER_HUB_USER', defaultValue: 'keshabyebye', description: 'Логин на Docker Hub')
    }

    environment {
        WORKSPACE_DIR = "${env.WORKSPACE}"
        APP_DIR = 'simple-java-maven-app'
```

```
        DOCKER_IMAGE =
"${params.DOCKER_HUB_USER}/${params.APP_NAME}:${params.APP_VER}"
    }

    stages {
        stage('Pre-check') {
            steps {
                script {
                    echo "Проверяем доступность сервера ${params.server}..."
                    sh "ping -c 2 ${params.server} || echo 'Ping не прошел, но пробуем
дальше'"
                }
            }
        }

        stage('Checkout') {
            steps {
                script {
                    echo "Клонируем репозиторий..."
                    sh "rm -rf ${APP_DIR}"
                    git branch: 'jenkins', url: 'https://github.com/SeigneurPuppy1/jenkins.git'
                }
            }
        }

        stage('Build') {
            steps {
                dir("${APP_DIR}") {
                    sh """
                    echo "=== СБОРКА MAVEN ПРОЕКТА ==="
                    mvn clean package -DskipTests
                    echo "JAR файлы:"
                    ls -la target/*.jar
                    """
                }
            }
        }

        stage('Test') {
            steps {
                script {
                    try {
                        echo "Запуск тестов..."
                        dir("${APP_DIR}") {
```

```
                    sh "mvn test"
                }
                echo "Тесты пройдены"
            } catch (Exception e) {
                echo "Тесты не пройдены: ${e.message}"
                error("Тесты не пройдены")
            }
        }
    }

    stage('Docker Build') {
        steps {
            script {
                echo "Создаем Docker образ..."
                dir("${APP_DIR}") {
                    echo "Собираем образ: ${DOCKER_IMAGE}"
                    sh """
                    docker build -t ${DOCKER_IMAGE} .
                    echo "Проверка образа:"
                    docker images | grep ${params.APP_NAME}
                    """
                }
            }
        }
    }

    stage('Docker Push to Hub') {
        steps {
            script {
                echo "Загружаем образ на Docker Hub..."
                withCredentials([usernamePassword(credentialsId: 'DOCKER_HUB',
usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                    sh """
                    echo "Выполняем вход в Docker Hub..."
                    echo "${DH_PASS}" | docker login -u "${DH_USER}" --password-
stdin

                    echo "Пушим образ ${DOCKER_IMAGE}..."
                    docker push ${DOCKER_IMAGE}

                    echo "Образ успешно загружен на Docker Hub!"
                    """
                }
```

```
            }
        }
    }

        stage('Deploy') {
        steps {
            script {
                echo "Развертывание на сервере ${params.server}..."

                withCredentials([usernamePassword(credentialsId: 'DOCKER_HUB',
usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                    sshagent(credentials: ['ssh_key']) {
                        sh """
                        echo "Подключаемся к серверу и скачиваем образ с Docker
Hub..."

                        ssh -o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null \\
                            little@${params.server} bash -s << 'EOF'
                            echo "Входим в Docker Hub на сервере..."
                            echo "${DH_PASS}" | docker login -u "${DH_USER}" --
password-stdin

                            echo "Скачиваем образ ${DOCKER_IMAGE} с Docker Hub..."
                            if ! docker pull ${DOCKER_IMAGE}; then
                                echo "ОШИБКА: Не удалось скачать образ с Docker Hub!"
                                exit 1
                            fi

                            echo "Останавливаем и удаляем старый контейнер (если
есть)..."
                            docker stop ${params.APP_NAME} 2>/dev/null || true
                            docker rm ${params.APP_NAME} 2>/dev/null || true

                            echo "Запускаем приложение..."
                            docker run \\
                                --name ${params.APP_NAME} \\
                                ${DOCKER_IMAGE}

                            echo "✅ Приложение выполнило свою задачу и
завершилось."
                            echo "Выходим из Docker Hub (очистка)..."
                            docker logout
EOF
```

```
                    """
                }
            }
        }
    }

    stage('Verify') {
        steps {
            script {
                try {
                    echo "Проверяем работу приложения..."
                    sshagent(credentials: ['ssh_key']) {
                        sh """
                        ssh -o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null \\
                            little@${params.server} bash -s << 'EOF'
                            sleep 5

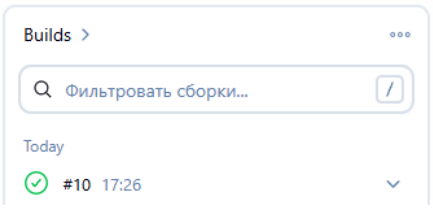
                            echo "Статус контейнера:"
                            docker ps -a | grep ${params.APP_NAME} || exit 1

                            echo "Логи приложения:"
                            docker logs ${params.APP_NAME} || true

                            if docker logs ${params.APP_NAME} 2>&1 | grep -q "Hello
World"; then
                                echo "Приложение работает корректно!"
                                exit 0
                            else
                                echo "Приложение запущено, но вывод отличается"
                                exit 0
                            fi
EOF
                        """
                    }
                } catch (Exception e) {
                    echo "Ошибка проверки: ${e.message}"
                    currentBuild.result = 'UNSTABLE'
                }
            }
        }
    }
}
```

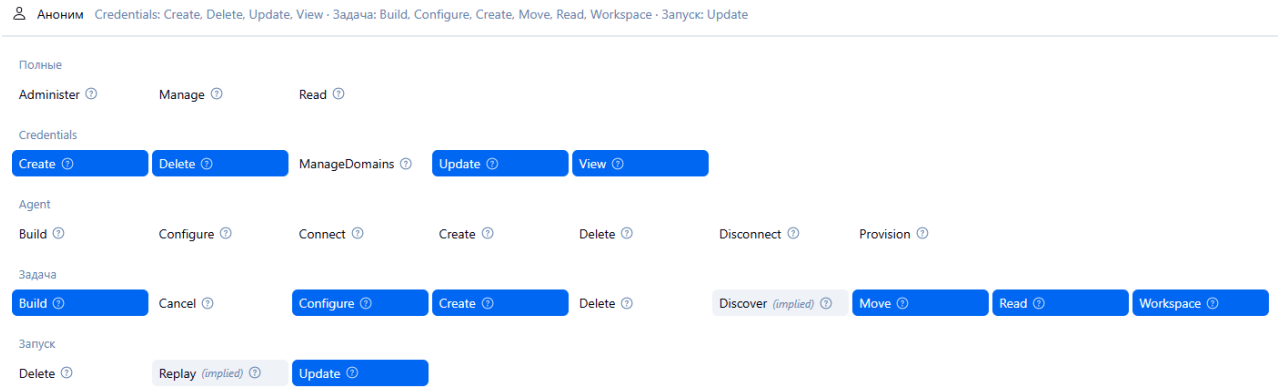
```
    post {
        always {
            echo "Пайплайн завершен. Статус: ${currentBuild.result ?: 'SUCCESS'}"
            junit allowEmptyResults: true, testResults: "${APP_DIR}/target/surefire-
reports/*.xml"
            cleanWs()
        }
        success {
            echo "Все этапы выполнены успешно!"
        }
        failure {
            script {
                echo "Пайплайн завершился с ошибками! Отправляю уведомление..."
                sh """
                MESSAGE="[ОШИБКА] Сборка \${JOB_NAME}
#\${BUILD_NUMBER}! Ссылка: \${BUILD_URL}"
                curl -s -X POST "https://api.vk.com/method/messages.send" \\
                  -d "peer_id=366737789" \\
                  -d "message=\${MESSAGE}" \\
                  -d "random_id=\$(date +%s)" \\
                  -d "access_token=${VK_ACCESS_TOKEN}" \\
                  -d "v=5.199" > /dev/null
                echo "Уведомление отправлено."
                """
            }
        }
    }
}
```

12. Теперь сделаем повторный пуш, например, создам пустой файл text и запушу на гитхаб. После этих действий запустилась джоба:

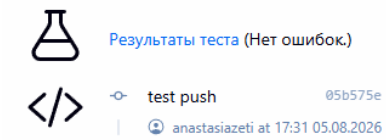


Но она упала с ошибкой, так как не смогла получить доступ к кредам для входа на docker hub, произошло потому что, запускается от другого пользователя, потому что при запуске напрямую все отрабатывает. Необходимо настроить доступ к кредам для пользователя от которого происходит запуск джобы после пуша

13. Для открытия доступа переходим в настройки дженкинс – security и находим вот такой пункт и устанавливаем как на скрине. Сохраняем и пробуем повторный пуш:



14.Вот результат работы:




```
Generic Cause
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/HW-32
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Pre-check)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Проверяем доступность сервера 62.60.246.108...
[Pipeline] sh
+ ping -c 2 62.60.246.108
PING 62.60.246.108 (62.60.246.108) 56(84) bytes of data.
64 bytes from 62.60.246.108: icmp_seq=1 ttl=59 time=39.1 ms
64 bytes from 62.60.246.108: icmp_seq=2 ttl=59 time=39.1 ms

--- 62.60.246.108 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 39.063/39.067/39.072/0.004 ms
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Клонируем репозиторий...
[Pipeline] sh
+ rm -rf simple-java-maven-app
```

```
Cloning the remote Git repository
Cloning repository https://github.com/SeigneurPuppy1/jenkins.git
 > git init /var/lib/jenkins/workspace/HW-32 # timeout=10
Получение изменений из вышестоящего репозитория https://github.com/SeigneurPuppy1/jenkins.git
 > git --version # timeout=10
 > git --version # 'git version 2.43.0'
 > git fetch --tags --force --progress -- https://github.com/SeigneurPuppy1/jenkins.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/SeigneurPuppy1/jenkins.git # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Избегайте повторной выборки
 > git rev-parse refs/remotes/origin/jenkins^{commit} # timeout=10
Проверка ревизии 05b575e130ef2268362c56694de6e14a7b1a4890 (refs/remotes/origin/jenkins)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 05b575e130ef2268362c56694de6e14a7b1a4890 # timeout=10
 > git branch -a -v --no-abbrev # timeout=10
 > git checkout -b jenkins 05b575e130ef2268362c56694de6e14a7b1a4890 # timeout=10
Сообщение о коммите: "test push"
 > git rev-list --no-walk 968935577b7b2011755c1bcc95ec55cfc8d5c39b # timeout=10
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Сборка)
[Pipeline] dir
Запуск в /var/lib/jenkins/workspace/HW-32/simple-java-maven-app
[Pipeline] {
[Pipeline] sh
+ echo === СБОРКА ПРОЕКТА MAVEN ===
=== СБОРКА ПРОЕКТА MAVEN ===
+ mvn clean package -DskipTests
[ [1;34mINFO [m] Сканирование проектов...
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m----------------------<  [0;36mcom.mycompany.app:my-app [0;1m >---------------------- [m
[ [1;34mINFO [m]  [1mСборка my-app 1.0-SNAPSHOT [m
[ [1;34mINFO [m]  [1m--------------------------------[ jar ]--------------------------------- [m
[ [1;34mINFO [m]
[ [1;34mINFO [m] ------------------------------------------------------
[ [1;34mINFO [m]  T E S T S
[ [1;34mINFO [m] ------------------------------------------------------
[ [1;34mINFO [m] Running com.mycompany.app. [1mAppTest [m
[ [1;34mINFO [m]  [1;32mTests run:  [0;1;32m2 [m, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.077 s - in com.mycompany.app. [1mAppTest [m
[ [1;34mINFO [m]
[ [1;34mINFO [m] Results:
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1;32mTests run: 2, Failures: 0, Errors: 0, Skipped: 0 [m
[ [1;34mINFO [m]
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m]  [1;32mBUILD SUCCESS [m
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
[ [1;34mINFO [m] Total time:  2.240 s
[ [1;34mINFO [m] Finished at: 2026-08-05T17:32:13+03:00
[ [1;34mINFO [m]  [1m------------------------------------------------------------------------ [m
```

```
 Sending build context to Docker daemon     64kB

 Step 1/5 : FROM eclipse-temurin:21-jre
  ---> 273396ed5998
 Step 2/5 : WORKDIR /app
  ---> Using cache
  ---> 2ac4dce300fa
 Step 3/5 : COPY target/*.jar app.jar
  ---> c7c874f7beda
 Step 4/5 : EXPOSE 8080
  ---> Running in 6fc6113d3359
  ---> Removed intermediate container 6fc6113d3359
  ---> a870b3b7a7cf
 Step 5/5 : ENTRYPOINT ["java", "-jar", "app.jar"]
  ---> Running in 6523958d015b
  ---> Removed intermediate container 6523958d015b
  ---> 8e1f61f9a790
 Successfully built 8e1f61f9a790
 Successfully tagged keshabyebye/maven-app:v01
 + echo Проверка образа:
 Проверка образа:
 + docker images
 + grep maven-app
 WARNING: This output is designed for human readability. For machine-readable output, please use --format.
 keshabyebye/maven-app:v01   8e1f61f9a790       454MB          115MB
Warning: A secret was passed to "sh" using Groovy String interpolation, which is insecure.
                 Affected argument(s) used the following variable(s): [DH_PASS]
                 See https://jenkins.io/redirect/groovy-string-interpolation for details.
+ echo Выполняем вход в Docker Hub...
Выполняем вход в Docker Hub...
+ echo ****
+ docker login -u keshabyebye --password-stdin
Login Succeeded
+ echo Пушим образ keshabyebye/maven-app:v01...
Пушим образ keshabyebye/maven-app:v01...
+ docker push keshabyebye/maven-app:v01
The push refers to repository [docker.io/keshabyebye/maven-app]
8720f40ba9a8: Pushed
v01: digest: sha256:8e1f61f9a7903082a48d2ee7aa305185ff56fc86cd83831f9143b3f81130c852 size: 2047
+ echo Образ успешно загружен на Docker Hub!
Образ успешно загружен на Docker Hub!
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
Развертывание на сервере 62.60.246.108...
[Pipeline] withCredentials
Masking supported pattern matches of $DH_PASS
[Pipeline] {
[Pipeline] sshagent
[ssh-agent] Using credentials jenkins
$ ssh-agent
SSH_AUTH_SOCK=/tmp/ssh-cNZW6t7Jc6OG/agent.447080
SSH_AGENT_PID=447084
Running ssh-add (command line suppressed)
[ssh-agent] Started.
[Pipeline] {
[Pipeline] sh
Warning: A secret was passed to "sh" using Groovy String interpolation, which is insecure.
                 Affected argument(s) used the following variable(s): [DH_PASS]
                 See https://jenkins.io/redirect/groovy-string-interpolation for details.
+ echo Подключаемся к серверу и скачиваем образ с Docker Hub...
```

```
+ ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null little@62.60.246.108 bash -s
Warning: Permanently added '62.60.246.108' (ED25519) to the list of known hosts.
Входим в Docker Hub на сервере...

WARNING! Your credentials are stored unencrypted in '/home/little/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
Скачиваем образ keshabyebye/maven-app:v01 с Docker Hub...
v01: Pulling from keshabyebye/maven-app
8720f40ba9a8: Pulling fs layer
8720f40ba9a8: Download complete
8720f40ba9a8: Pull complete
Digest: sha256:8e1f61f9a7903082a48d2ee7aa305185ff56fc86cd83831f9143b3f81130c852
Status: Downloaded newer image for keshabyebye/maven-app:v01
docker.io/keshabyebye/maven-app:v01
Останавливаем и удаляем старый контейнер (если есть)...
maven-app
maven-app
Запускаем приложение...
Hello World!
✅ Приложение выполнило свою задачу и завершилось.
Выходим из Docker Hub (очистка)...
Removing login credentials for https://index.docker.io/v1/
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 447084 killed;
[ssh-agent] Stopped.
Проверяем работу приложения...
[Pipeline] sshagent
[ssh-agent] Using credentials jenkins
$ ssh-agent
SSH_AUTH_SOCK=/tmp/ssh-hV9PGJJS1f0f/agent.447106
SSH_AGENT_PID=447110
Running ssh-add (command line suppressed)
[ssh-agent] Started.
[Pipeline] {
[Pipeline] sh
+ ssh -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null little@62.60.246.108 bash -s
Warning: Permanently added '62.60.246.108' (ED25519) to the list of known hosts.
Статус контейнера:
a6a15b91d0e1   keshabyebye/maven-app:v01   "java -jar app.jar"   6 seconds ago   Exited (0) 6 seconds ago            maven-app
Логи приложения:
Hello World!
Приложение работает корректно!
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 447110 killed;
[ssh-agent] Stopped.
[Pipeline] // sshagent
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Пайплайн завершен. Статус: SUCCESS
[Pipeline] junit
Recording test results
```

```
[Checks API] No suitable checks publisher found.
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] done
[Pipeline] echo
Все этапы выполнены успешно!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

15. Вот переделанный скрипт с учетом задания для параллельного запуска. Запускает две ветви: Build & Test – mvn ckean package –T 1C (собирает и гоняет тесты). Code Check mvn validate проверка структуры проекта:

```
pipeline {
    agent any

    parameters {
        string(name: 'server', defaultValue: '62.60.246.108', description: 'IP сервера для запуска приложения')
        string(name: 'APP_NAME', defaultValue: 'maven-app', description: 'Название приложения')
        string(name: 'APP_VER', defaultValue: 'v01', description: 'Версия приложения')
        string(name: 'DOCKER_HUB_USER', defaultValue: 'keshabyebye', description: 'Логин на Docker Hub')
    }

    environment {
        WORKSPACE_DIR = "${env.WORKSPACE}"
        APP_DIR = 'simple-java-maven-app'
        DOCKER_IMAGE = "${params.DOCKER_HUB_USER}/${params.APP_NAME}:${params.APP_VER}"
    }

    stages {
        stage('Pre-check') {
            steps {
                script {
                    echo "Проверяем доступность сервера ${params.server}..."
                    sh "ping -c 2 ${params.server} || echo 'Ping не прошел, но пробуем дальше'"
                }
            }
        }

        stage('Checkout') {
            steps {
                script {
                    echo "Клонируем репозиторий..."
                    sh "rm -rf ${APP_DIR}"
                    git branch: 'jenkins', url: 'https://github.com/SeigneurPuppy1/jenkins.git'
                }
            }
        }
```

```
        stage('Parallel Build') {
            parallel {
                stage('Build & Test') {
                    steps {
                        dir("${APP_DIR}") {
                            sh """
                            echo "=== СБОРКА И ТЕСТЫ ==="
                            mvn clean package -T 1C
                            echo "JAR файлы:"
                            ls -la target/*.jar
                            """
                        }
                    }
                }

                stage('Code Check') {
                    steps {
                        dir("${APP_DIR}") {
                            sh """
                            echo "=== ПРОВЕРКА КОДА ==="
                            echo "Проверка pom.xml..."
                            mvn validate

                            echo "Структура проекта:"
                            tree -L 3 || find . -type f -name "*.java" | head -10

                            echo "Количество Java-файлов:"
                            find src -name "*.java" | wc -l

                            echo "Проверка завершена"
                            """
                        }
                    }
                }
            }
            post {
                always {
                    junit allowEmptyResults: true, testResults: "${APP_DIR}/target/surefire-
reports/*.xml"
                }
            }
        }

        stage('Docker Build') {
```

```
        steps {
            script {
                echo "Создаем Docker образ..."
                dir("${APP_DIR}") {
                    echo "Собираем образ: ${DOCKER_IMAGE}"
                    sh """
                    docker build -t ${DOCKER_IMAGE} .
                    echo "Проверка образа:"
                    docker images | grep ${params.APP_NAME}
                    """
                }
            }
        }
    }

    stage('Docker Push to Hub') {
        steps {
            script {
                echo "Загружаем образ на Docker Hub..."
                withCredentials([usernamePassword(credentialsId: 'DOCKER_HUB',
usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                    sh """
                    echo "Выполняем вход в Docker Hub..."
                    echo "${DH_PASS}" | docker login -u "${DH_USER}" --password-
stdin

                    echo "Пушим образ ${DOCKER_IMAGE}..."
                    docker push ${DOCKER_IMAGE}

                    echo "Образ успешно загружен на Docker Hub!"
                    """
                }
            }
        }
    }

    stage('Deploy') {
        steps {
            script {
                echo "Развертывание на сервере ${params.server}..."

                withCredentials([usernamePassword(credentialsId: 'DOCKER_HUB',
usernameVariable: 'DH_USER', passwordVariable: 'DH_PASS')]) {
                    sshagent(credentials: ['ssh_key']) {
```

```
                    sh """
                    echo "Подключаемся к серверу и скачиваем образ с Docker
Hub..."

                    ssh -o StrictHostKeyChecking=accept-new little@${params.server}
bash -s << 'EOF'
                      echo "Входим в Docker Hub на сервере..."
                      echo "${DH_PASS}" | docker login -u "${DH_USER}" --
password-stdin

                      echo "Скачиваем образ ${DOCKER_IMAGE} с Docker Hub..."
                      if ! docker pull ${DOCKER_IMAGE}; then
                        echo "ОШИБКА: Не удалось скачать образ с Docker Hub!"
                        exit 1
                      fi

                      echo "Останавливаем и удаляем старый контейнер (если
есть)..."
                      docker stop ${params.APP_NAME} 2>/dev/null || true
                      docker rm ${params.APP_NAME} 2>/dev/null || true

                      echo "Запускаем приложение..."
                      docker run -d --name ${params.APP_NAME} -p 8080:8080
${DOCKER_IMAGE}

                      echo "✅ Контейнер запущен"
                      docker logout
EOF
                    """
                  }
                }
              }
            }
          }

          stage('Verify') {
            steps {
              script {
                try {
                  echo "Проверяем работу приложения..."
                  sshagent(credentials: ['ssh_key']) {
                    sh """
                    ssh -o StrictHostKeyChecking=accept-new little@${params.server}
bash -s << 'EOF'
```

```
                    sleep 5

                    echo "Статус контейнера:"
                    docker ps -a | grep ${params.APP_NAME} || exit 1

                    echo "Логи приложения:"
                    docker logs ${params.APP_NAME} || true

                    if docker logs ${params.APP_NAME} 2>&1 | grep -q "Hello
World"; then
                        echo "Приложение работает корректно!"
                        exit 0
                    else
                        echo "Приложение запущено, но вывод отличается"
                        exit 0
                    fi
EOF
                """
                }
            } catch (Exception e) {
                echo "Ошибка проверки: ${e.message}"
                currentBuild.result = 'UNSTABLE'
            }
        }
    }
  }

  post {
    always {
      echo "Пайплайн завершен. Статус: ${currentBuild.result ?: 'SUCCESS'}"
      cleanWs()
    }
    success {
      echo "Все этапы выполнены успешно!"
    }
    failure {
      script {
        echo "Пайплайн завершился с ошибками! Отправляю уведомление..."
        sh """
        MESSAGE="[ОШИБКА] Сборка \${JOB_NAME}
#\${BUILD_NUMBER}! Ссылка: \${BUILD_URL}"
        curl -s -X POST "https://api.vk.com/method/messages.send" \\
          -d "peer_id=366737789" \\
```

```
                -d "message=\${MESSAGE}" \\
                -d "random_id=\$(date +%s)" \\
                -d "access_token=${VK_ACCESS_TOKEN}" \\
                -d "v=5.199" > /dev/null
            echo "Уведомление отправлено."
            """
        }
    }
  }
}
```

16. Результат работы пайплайна с параллельным запуском. Посмотреть можно перейдя в номер сборки и зайти в пункт Pipeline Overview, просто как наглядный пример отработки параллельной работы:

